

Implementing and optimising Lenia and the effect of growth function parameters on world outcome.

Head of Class: David Buscher

Abstract

The cellular automaton Lenia is implemented in Python, making use of NumPy and Numba to compile code into lower-level languages to improve efficiency. The computational complexities of multiple Lenia algorithms are predicted and verified experimentally. The relationship between growth function parameters and world outcome is investigated by mapping the outcome of worlds at many parameter choices. Based on this investigation, a rule for determining the outcome of a world from the parameter choices is proposed. The implementation is verified by successfully recreating results from previous researchers. The methods of: FFT convolution, NumPy vectorisation, and Numba compiling are used and each one is found to significantly improve efficiency.

(Word count: Abstract: 107 words, Report: 2911 words)

1.1 Introduction

Lenia is a *cellular automaton* – an array of cells that update iteratively following microscopic rules to produce emergent phenomenon at the macroscopic scale. It can be thought of as a generalisation of the famous Game of Life devised by John Conway in 1970 [1]. Lenia was devised by Bert Chan in 2019 [2], who discovered and demonstrated many of the interesting emergent phenomena such as stable, self-propelling, interacting “life” – patterns that present some characteristics reminiscent of biological life. *Continuous Lenia* is the theoretical continuum limit of the cellular automaton *Discrete Lenia*, the latter of which is the subject of this project.

1.2 Definitions of Lenia

Lenia has three key mathematical objects that define it: the *world*, *site set*, and *local rule*. The *world* is an array of sites, in this project it is two-dimensional, discrete, and square. All implementations of Lenia have a finite number of discrete sites but approach the theoretical *Continuous Lenia* as the discretisation goes to zero and the number of sites goes to infinity. In Lenia each site takes a real value between zero and one, making the *site set* the real numbers in that range. While technically discretised by the precision of the floating-point data type, the site set is effectively continuous. The *local rule* is the function applied at each site, taking the site's value and the values of its neighbourhood as inputs and outputting the updated site value. The total action of applying the local rule at each site in the world is called the *global rule*, and in Lenia is equivalent to the convolution of the *world* and *kernel* (discussed later). As we will only ever apply the global rule and never individual local rules, these terms are used interchangeably in this paper. Each application of the global rule progresses the system by one time step, dt , and like the world array, real Lenia has finite dt , but approaches Continuous Lenia as $dt \rightarrow 0$.

The local/global rule has three steps: first, the world is convolved with the kernel to produce the *potential* array, next the potential array is fed through the *growth function* element-wise, and finally summed with the original world array. This is summarised in Figure 1. In Lenia, the

kernel is a Gaussian Bump, and the growth function is a translated Gaussian (Figure 2). The only length scales in Lenia are the side length of the world, N , and the side length of the kernel, k . As the world has periodic boundary conditions and is significantly larger than the kernel, the length scale of the system is effectively defined by the user via the size of the kernel. For this reason, the continuum limit of Lenia is the limit as $k \rightarrow \infty$. Despite our inability to reach this continuum limit, in each case of discretisation complex and interesting behaviours can be found at reasonable degrees of discretisation, which is useful in keeping the computational expense manageable.

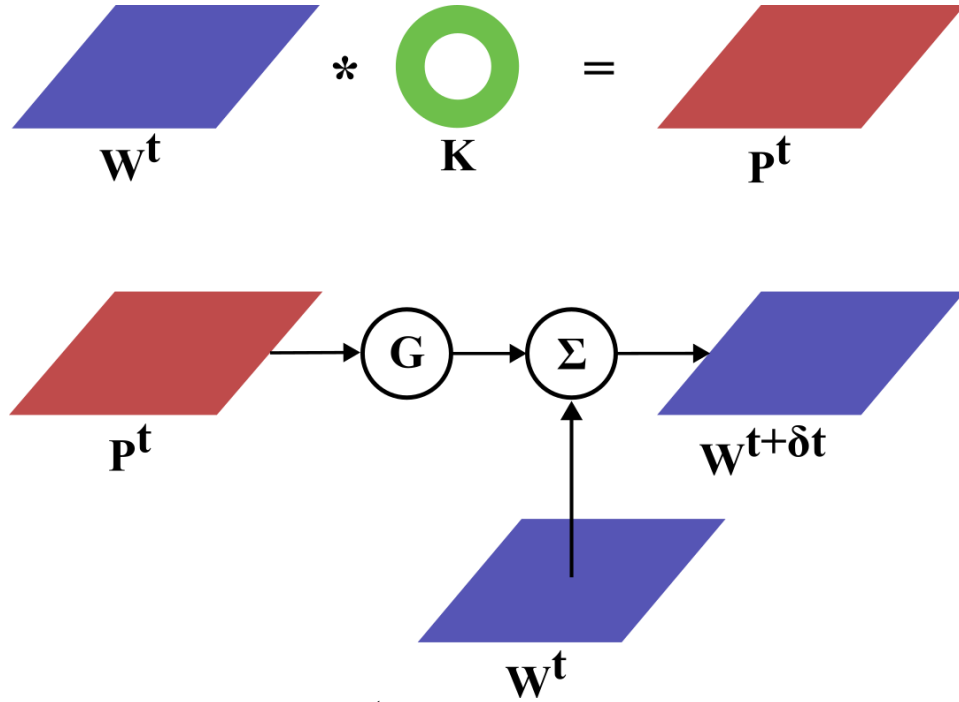


Figure 1 – The global rule of Lenia. W^t is the world array at time t , K is the kernel, P^t is the potential array at time t , G is the growth function and Σ denotes a sum of inputs. $W^{t+\delta t}$ is the world array progressed by one time step dt . $*$ denotes 2D convolution.

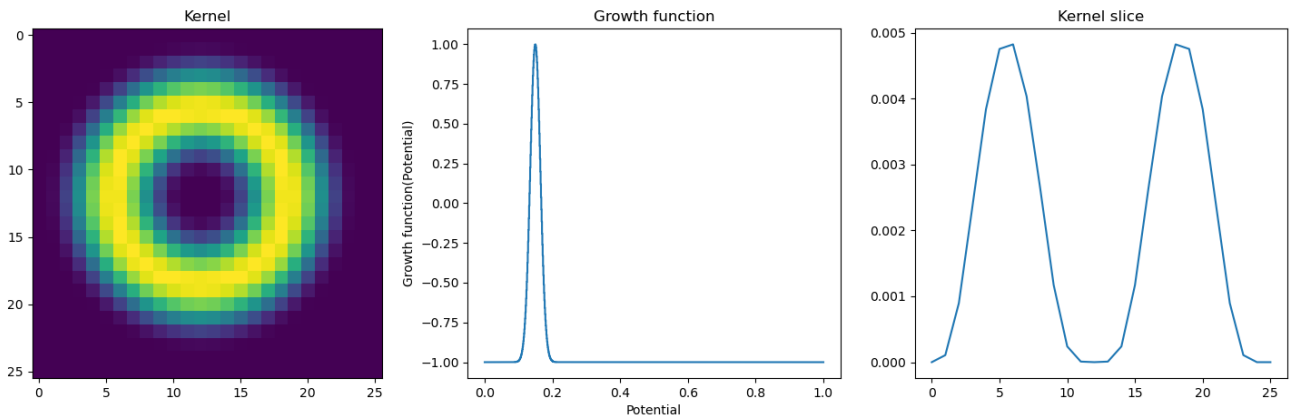


Figure 2 – (Left to right) The kernel, growth function, and cross-section of kernel of Lenia.

2.1 Complexity Analysis

The most computationally expensive step (as shown below) of the local rule is the two-dimensional convolution. Two possible algorithms for 2D convolution arise: direct convolution and convolution via Fourier Transforms following the convolution theorem [3].

The complexity of 2D convolution via the convolution theorem using Fourier Transforms depends on the complexity of the underlying Fourier Transform algorithm, for which the obvious choice is the Fast Fourier Transform, with a 1D complexity of $N \log N$ for N data points [4]. To find the complexity of the 2D convolution used in Lenia, we start with the convolution theorem:

$$\begin{aligned}\mathcal{F}(h * g) &= \mathcal{F}(h) \times \mathcal{F}(g) \\ \Rightarrow h * g &= \mathcal{F}^{-1}(\mathcal{F}(h) \times \mathcal{F}(g))\end{aligned}$$

where $\mathcal{F}$ means (Fast) Fourier Transform, $\mathcal{F}^{-1}$ Inverse Fourier Transform, and $*$ denotes convolution. In Lenia, h is the kernel, which is constant, and so $\mathcal{F}(h)$ is only calculated once, and the processing time is negligible over the many frames calculated. g is the world array. The complexity of a 2D FFT is:

$$1D \text{ FFT complexity: } O(N \log N)$$

$$2D \text{ FFT complexity: } O(N \log N) + O(N \log N) = O(N^2 \log N)$$

where N is now the side length of the 2D array. And as the 2D IFT has the same complexity, the overall complexity of the convolution is:

$$O(N^2 \log N)$$

The complexity of each step in the local rule is summarised in Table 1 below.

Process	Complexity, $O()$
2D Direct Convolution	$N^2 k^2$
2D Convolution via FFT	$N^2 \log N$
Growth Function	N^2
Sum	N^2

Table 1 – Computational Complexity of each of the steps of Lenia’s Local Rule. The complexities of two methods of convolution are included and it is seen that for values of N relevant to this project the 2D FFT method is faster.

As in both cases the convolution is the most complex step, the overall complexity of Lenia is equal to the complexity of the convolution process. For all cases in this project the side length of the kernel is $k=26$, chosen according to Chan 2021 [5]. $N^2 k^2$ (Direct) is larger than $N^2 \log N$ (FFT) for all $N < \exp(k^2)$. With $k=26$ this is $N_{\max} \approx 10^{293}$, clearly well above the limits of our computation. Therefore, we predict that for all relevant N the Fourier Transform method will be faster and will scale with N as $N^2 \log N$. These predictions are tested in Section 4.3.

3.1 Implementation

This project was implemented in Python, making use of NumPy and later Numba for improving processing speed, and matplotlib and IPython for display. Multiple versions of the core Lenia algorithm were written to interface with different external packages. Initially, code was written specifically for use with matplotlib's `FuncAnimation` package. This requires a function to iteratively update the frame, which in Lenia is the local rule. The potential is computed via one of three methods: direct convolution, convolution via FFTs, and using SciPy's built-in FFT convolution function. Direct convolution is predicted (as in Section 2.1) to be slowest, and while SciPy's function and the custom written FFT convolution function both use the same method, SciPy wastes time recalculating the FFT of the unchanged kernel every time step, where the custom code only computes the kernel FFT once. Wherever possible, the code is vectorised to run in the faster NumPy C environment. This is achieved for each step of the local rule, as each of these steps has a suitable NumPy function, but not across time steps, as the local rule as a whole is not vectorizable through NumPy. Display is handled by `FuncAnimation`, which passes an animation to IPython's display package that displays an interactive HTML object. This display process turns out to take significantly more computing time than the underlying Lenia algorithm. For this reason, for parts of the project where animations aren't explicitly necessary, or while assessing the speed of the algorithm, animations were not rendered and only the last frame is displayed. Each frame is still computed, but it is not stored and is overwritten by the next frame.

To address the problem of still having to run the frame iterations in pure Python, Numba is used to compile the time-step function into fast machine code using the LLVM compiler library. Numba is able to both improve the speed of NumPy functions by compiling them further to machine code, and to improve the speed of the iteration function by avoiding the use of pure Python. Numba is, at the time of writing, still under significant development, and so many of the NumPy functions used in this project's original implementation of Lenia were not supported and had to be re-written by hand. Although this appears to introduce less efficient for-loops into the code, these functions are compiled by Numba, and so run as or more efficiently than their corresponding NumPy functions.

3.2 Parameter Map

Lenia can support stable patterns called *solitons* given suitable rule parameters. The relationship between the parameters of the growth function and the outcome of a world were investigated. The Lenia growth function, $G(U)$, where U is the potential:

$$G(U) = 2e^{-\frac{1}{2}\left(\frac{U-\mu}{\sigma}\right)^2} - 1$$

has two parameters: the mean μ and the standard deviation σ . The outcome of the world simulation was investigated by observing if an initially random world led to a "uniform desert" (the whole world is "dead" with cell states of zero) within 150 frames (chosen for convenience). The outcome was investigated over the 2D μ - σ parameter space in the range $\mu \in [0.1, 0.5]$, $\sigma \in [0, 0.12]$ and is plotted in Figure 4. This code took a long time to complete, and would benefit greatly from parallelisation, ideally on a GPU, as each world of different parameters can be simulated independently. An attempt was made to run this code on a GPU using Nvidia's CUDA framework and Numba, but this proved beyond the scope of this project.

This project was approached with the aim of writing resilient, customisable, and easy-to-read code. This was achieved by writing independent functions that perform general tasks and can

be applied to each specific instance of Lenia. It was decided not to use classes as they would have only marginally improved readability at the expense of additional overhead. Classes would also not have been compatible with Numba implementation, that only interprets basic Python functionalities. When adapting certain algorithms for compiling in Numba, readability and customisability occasionally had to be sacrificed in order to produce a code-structure correctly readable by the compiler, but this was minimised by leaving as much code in general functions as possible. Auxiliary functions for tasks such as plotting the kernel/growth function or computing the FFT of the kernel were written to reduce the amount of code and user input needed for each specific instance of Lenia. A user wishing to render a customised instance of Lenia, for example, would only have to provide the kernel, growth function, and details of the initial state and would be able to use generic functions to compute Lenia.

4.1 Results

This project successfully produced a lightweight and efficient implementation of Lenia, capable of rendering large worlds for many frames. Using the Numba algorithm, a one thousand by one thousand world can be simulated for 150 frames in under 30 seconds on mid-range hardware. This implementation of Lenia was tested by verifying that it produces the expected results in cases studied by previous researchers. The first such case is a specific subset of Lenia – Conway's of Game Life (GoL) which has a small and discrete kernel and a step growth function. This implementation successfully recreated GoL, displaying many of the key features such as stable solitons and self-propelling "gliders". An animation of GoL produced by this algorithm is included in the attached files. This implementation was further tested by checking for the existence of stable "life forms" in full Lenia. These entities have been investigated and demonstrated by the original author of Lenia and are recreated from manual patterns. *Orbium* is pictured below and animated in the attached notebook. An animation starting from a random world with growth function parameters suitable for *Orbium* is also included, to demonstrate the patterns that *Orbium* is capable of emerging randomly from.

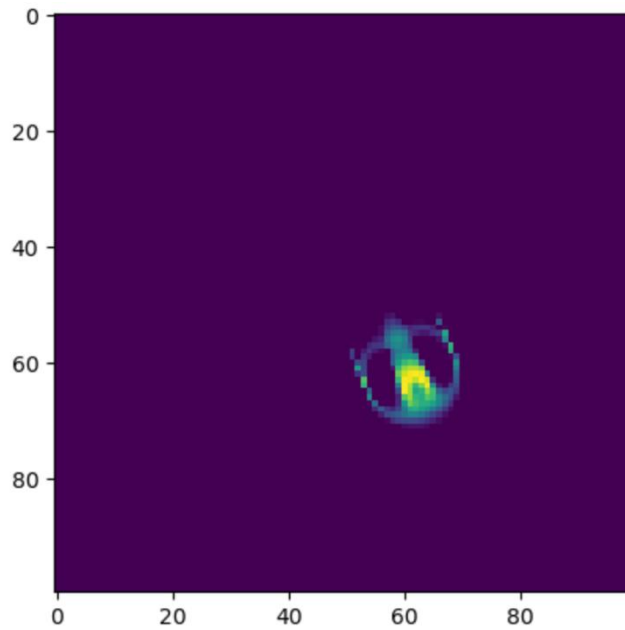


Figure 3 – Orbium, a “lifeform” first discovered by Chan 2019 and recreated here to verify the implementation of Lenia. Orbium is self-propelling (in this image it would travel down and to the right). The animation from which this image is taken is included in the attached files.

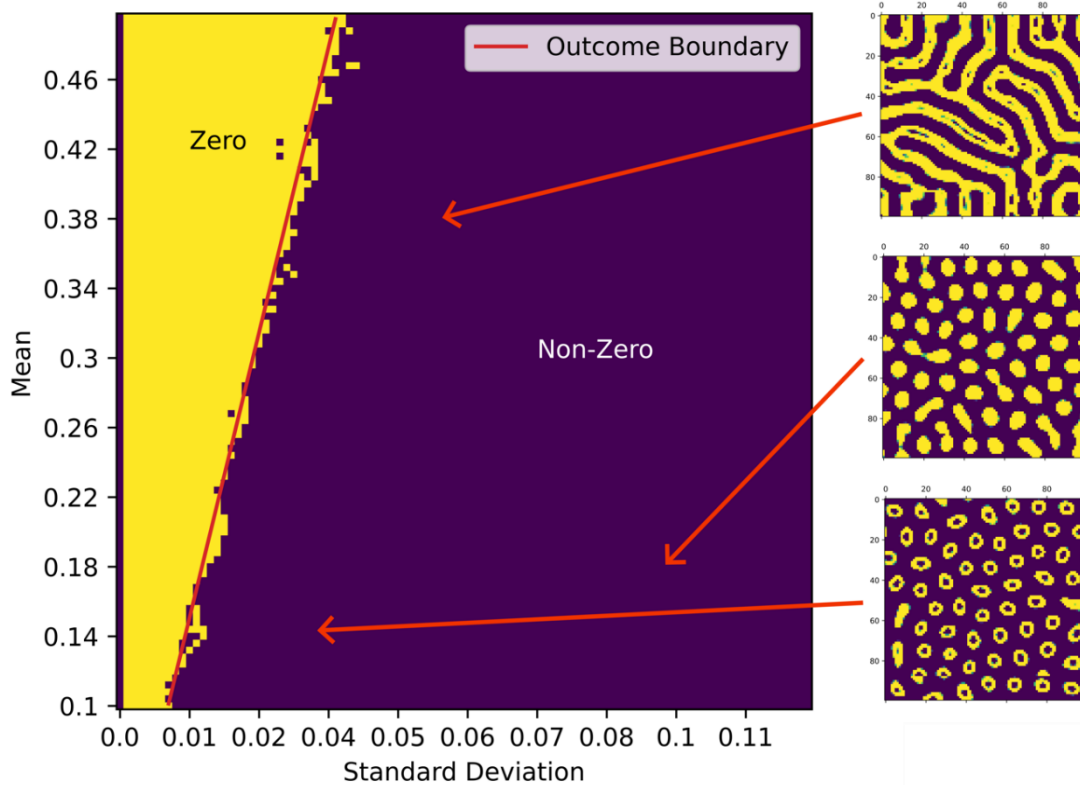


Figure 4 – μ - σ parameter map showing regions of different outcomes after 150 frames. The region left of outcome boundary corresponds to the “uniform desert” – all cells in the world are zero. All worlds right of the boundary show non-zero outcomes. Three non-zero worlds are included. The world at $\sigma=0.04$, $\mu=0.14$ (bottom right) shows the species Circium, also discovered by Chan 2019.

4.2 Parameter Map

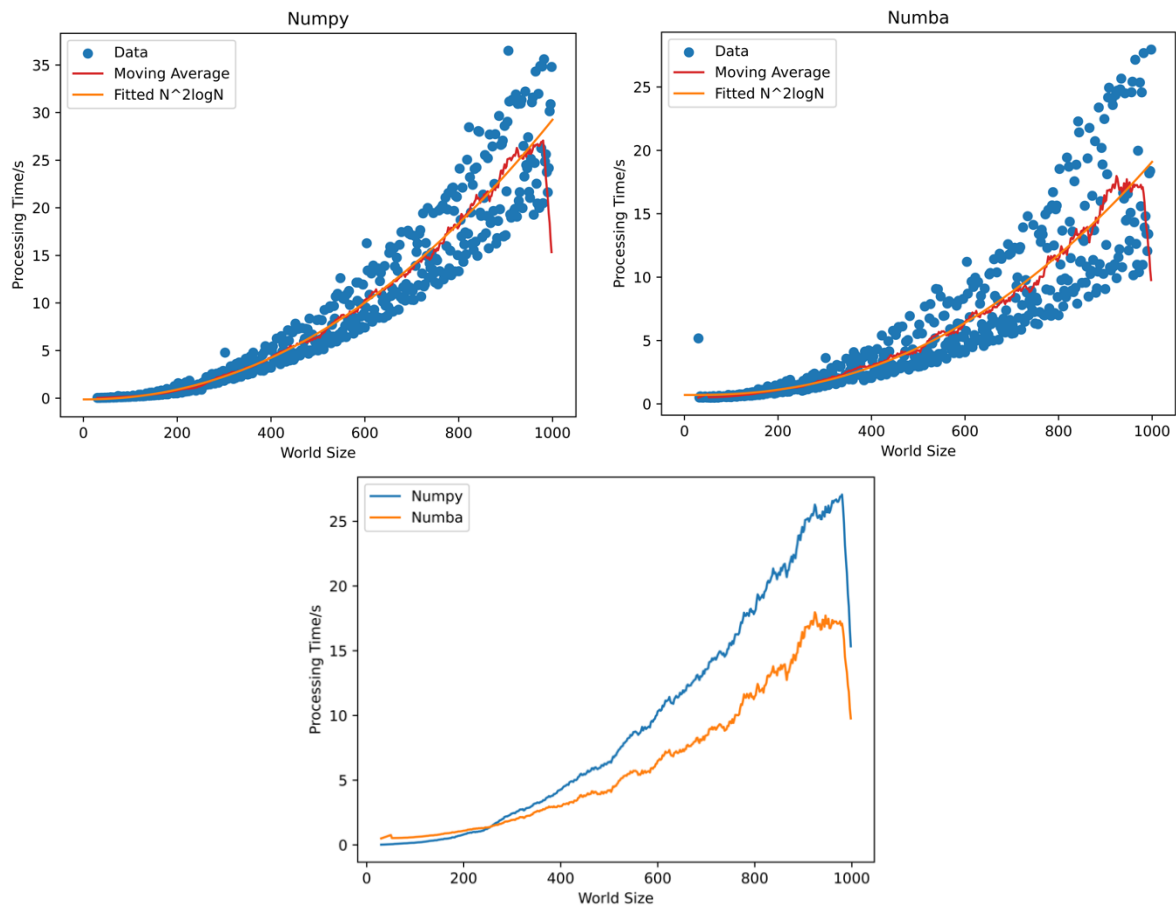
Lenia was found to produce worlds that were non-zero for at least 150 frames in the region right of the outcome boundary, which, in the parameter region investigated, suggests the rule:

$$outcome = \begin{cases} \text{zero (desert)} & \mu > 0.1\sigma + 0.46 \\ \text{nonzero} & \mu < 0.1\sigma + 0.46 \end{cases}$$

This rule presents a useful tool for further work with Lenia, as knowing which parameter choices should give rise to non-zero outcomes might be useful in troubleshooting future implementations.

4.3 Complexity Testing

The complexity of Lenia was predicted in Section 2.1 to scale as $N^2 \log N$ when the FFT method is used. This relation was tested by plotting the processing time against N for both NumPy calculation and Numba calculation in Figures 5-6. A regression analysis was done on both the raw data and moving averages of both methods, and in both cases the prediction is found to fit the data well, as summarised in Table 2.

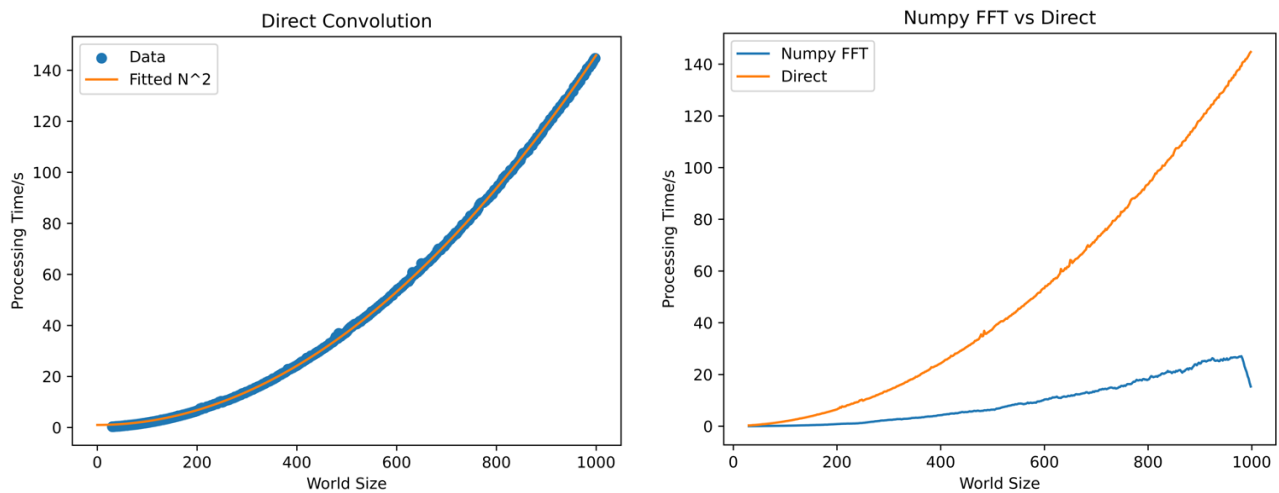


Figures 5-7 – Plots of processing time against N , with a moving average and fitted predicted curve. Numba is seen to perform faster than NumPy at large N . Both methods are found to follow a complexity relation of $O(N^2 \log N)$, as predicted. The sharp drop in the moving average at the right of each plot is an artifact of the convolution method used calculate this average.

Method	Data	R^2
NumPy FFT	Raw data	0.91
	Moving average	0.98
Numba FFT	Raw data	0.78
	Moving average	0.97
Direct	Raw data	0.9999

Table 2 – Coefficients of determination from a regression analysis of each convolution method's complexity relation. The moving average of each method is found to agree with prediction very well. The raw data, particularly of Numba, agrees less well due to the large spread of values as seen in Figure 6.

The complexity relation of direct convolution is also tested with a similar method, and the plot and regression analysis showed a good agreement with prediction. The benefit of FFT convolution over direct is demonstrated by the plot of both method's processing times (Figure 9).



Figures 8-9 – Plots of processing time against N for the direct convolution method, and comparing direct convolution and FFT convolution. The direct method is found to follow a complexity relation of $O(N^2)$ as predicted, and the FFT method is substantially faster than the direct method, in the N range investigated.

Although the complexity of the NumPy and Numba methods are found to be identical, as predicted, they are still free to differ by a numerical factor. This was investigated by plotting the moving averages of the N -scaling plot calculated previously for both NumPy and Numba, in Figure 7. We see that NumPy and Numba perform significantly differently in the N range investigated. At low N NumPy performs faster, but after a crossover at $N \approx 250$, Numba performs significantly faster, as predicted. The fitted complexity relation has a non-zero y intercept¹ suggesting that Numba introduces a processing constant independent of N but scales with N slower than NumPy.

¹ The relation approaches a non-zero value (~ 0.7 s) as $N \rightarrow 0$ but technically is undefined at $N=0$

5.1 Further Work:

Areas where this project could be extended are primarily in further improvements to the efficiency of the code, and in the development of the parameter space search. The processing time of the Lenia algorithm could be improved by parallelising certain processes. The most applicable would be the processing of the parameter map, as each data point is completely independent. The greatest improvement to the base algorithm might come from parallelising the convolution calculation, which would be possible by using direct convolution instead of by FFT, as each step of a convolution is independent. Whether parallelised direct convolution would be faster than FFT convolution with the number of cores available in consumer GPUs is an important question for further work. More efficient code would allow a greater breadth of research in parameter space or other fields. Further development of the parameter map, beyond parallelisation, could include a more detailed search algorithm, so that more than zero vs non-zero outcomes can be distinguished. A potentially interesting such algorithm would be a neural network pattern identifier capable of detecting and categorising recurring solitons. This would provide a method for automated species identification. The proposed rule for world-outcome should be verified and investigated at different numbers of frames, to determine to what extent it generally predicts the outcome of Lenia.

6.1 Environment:

All of the timed results in this project were completed with the following hard/software:

- Hardware: Intel i7-6700k
- Software: Python 3.11.5, Windows 10, NumPy 1.24.3, Numba 0.57.0, Rocket-FFT 0.2.5, SciPy 1.11.1

Some other tests were completed on a MacBook Pro (Intel i5, Monterey 12.6.7, Python and all packages as above).

7.1 Conclusion:

In this project we implemented an algorithm to compute the cellular automaton Lenia in Python. The complexities of multiple possible algorithms were derived to decide on the best choice. These predicted complexities were tested and a good agreement with data was found in all cases. NumPy and Numba were used to compile pure Python into faster C and machine code to improve the speed of the algorithm. The Numba algorithm was found to be significantly faster at large world sizes. The implementation of Lenia was tested by reproducing results from previous research, including Conway's Game of Life and the species *Orbium* and *Circium*. The relationship between growth function parameters and world outcome were investigated to find which sets of parameters produce non-zero outcomes. A rule for this relationship between parameters and outcome was proposed as a tool for future researchers. Further work to build on this project could include further improvements to code efficiency, or developments of the parameter space map.

8.1 Appendix: Description of code structure

The files attached with this report are:

<i>File Name</i>	<i>Description</i>
Numpy Lenia.ipynb	Jupyter Notebook containing NumPy Lenia
Numba Lenia.ipynb	Jupyter Notebook containing Numba Lenia
Numpy Lenia_python.py	Numpy Lenia.ipynb converted to .py file
Numba Lenia_python.py	Numba Lenia.ipynb converted to .py file
orbium.mp4	Animation of the species <i>Orbium</i> , referenced in Figure 3
lenia_demo.mp4	Animation of Lenia, referenced in Section 4.1
gol_demo.mp4	Animation of Conway's Game of Life, referenced in section 4.1
N_set_XXXXX.csv (5 files)	Comma delimited text files containing the set of N values tested where XXXXX is the method name
time_set_XXXXX.csv (5 files)	Comma delimited text files containing the set of time values tested where XXXXX is the method name
map_array.npy	NumPy file containing the array of outcome codes for the parameter map

This project was completed exclusively in Jupyter Notebooks and is intended to be used as such, a regular python file is included.

8.2 NumPy Lenia

The code for Lenia using NumPy is in the notebook “Numpy Lenia.ipynb”. It has the required packages:

<i>Package</i>	<i>Nickname in project</i>	<i>Description</i>
NumPy	np	Workhorse of this implementation, runs key functions in faster C environment.
matplotlib.pyplot	plt	Used for plotting of timing data and parameter map.
matplotlib.animation	animation	Used for animation of Lenia.
IPython.display.HTML	HTML	Used to display Lenia animations as interactive HTML objects.
scipy.signal	signal	Used to compute 2D FFT Convolution, for testing only.
itertools	it	Used for iterating over multiple iterables.
warnings	warnings	Used to raise warnings to the user.
time.gmtime, strftime	gmtime, strftime	Used for naming saved files.
time.perf_counter	perf_counter	Used for timing functions.
scipy.optimize	sc_opt	Used to fit the predicted complexity curve to experimental data.

The first functions defined in the Numpy Lenia project are utility functions for displaying animations, displaying the kernel and growth function, and computing the kernel’s FFT.

Next, Lenia’s kernel, growth function, and local rule are defined using NumPy’s FFT function. Lenia is then animated.

Various functions for use in computing the parameter map are defined, most importantly find_outcome that returns the outcome of a world within a specified number of frames.

The scaling relation with N is investigated using the iterator function.

The parameter map is computed using the previously mentioned functions.

8.3 Numba Lenia

The code for Lenia using Numba compiling is in the notebook “Numba Lenia.ipynb”. It has the required packages:

<i>Package</i>	<i>Nickname in project</i>	<i>Description</i>
NumPy	np	Workhorse of this implementation, runs key functions in faster C environment.
matplotlib.pyplot	plt	Used for plotting of timing data and parameter map.
time.perf_counter	perf_counter	Used for timing functions.
scipy.optimize	sc_opt	Used to fit the predicted complexity curve to experimental data.
numba.jit	jit	Decorator used to call Numba to compile Python code into machine language.
rocket_fft	N/A	An auxiliary package that makes Numba aware of NumPy's 2D FFT function and allows it to compile it.

Each task requires the Lenia local rule to be re-defined to allow Numba to compile it in each case. The Numba code is therefore less readable than the NumPy code, but follows a very similar structure.

References:

[1] – Gardner, Martin. "The Fantastic Combinations of John Conway's New Solitaire Game "Life." Sc. Am. 223 (1970): 20-123.

[2] – Chan, Bert Wang-Chak. "Lenia-biology of artificial life." arXiv preprint arXiv:1812.05433 (2018).

[3] – Cowley, S. J. "NST IB Math. Methods I Section 3c – Fourier Transforms". NST 1B Maths, Nov. 2022, University of Cambridge.

[4] – Cooley, James W., and John W. Tukey. "An algorithm for the machine calculation of complex Fourier series." Mathematics of computation 19.90 (1965): 297-301.

[5] Chan, Bert Wang-Chak. "From Conway to Lenia" GitHub repository (2021) Accessed at <https://github.com/OpenLenia/Lenia-Tutorial> on 25/4/2024